

D-SafeMPC: Diffusion-Driven Safe Model Predictive Control with Discrete-Time Control Barrier Functions

Erdi Sayar¹, Ersin Daş², Joel W. Burdick³, Alois Knoll⁴, and Erdal Kayacan¹

Abstract—A key limitation on the use of diffusion models in robotic planning is their inability to inherently enforce safety or dynamical constraints, which often results in physically infeasible or unsafe outputs. Hybrid approaches that employ model predictive control (MPC) to address this problem can be unstable, as poor trajectory initializations from the diffusion model prevent the MPC from converging to a safe and feasible solution. To overcome these challenges, we propose D-SafeMPC, which enhances the interaction between diffusion and control. Our method guides the reverse diffusion process with control barrier functions (CBFs) and control Lyapunov functions (CLFs) and employs an iterative-projection scheme where an MPC refines the trajectory at each denoising step. This steers sampling toward safe, goal-directed regions and provides reliable MPC warm starts. In simulations on a Franka manipulator across four scenarios (one static-obstacle and three dynamic-obstacle settings) and in a sim-to-real experiment on a physical Franka robot, D-SafeMPC improves safety, task success rates, and planning efficiency over state-of-the-art baselines. To facilitate reproducibility, our source code and experimental configurations are available in a repository at <https://github.com/erdiphd/D-SafeMPC>

I. INTRODUCTION

Diffusion models [1] have established state-of-the-art performance in generative tasks such as image and video synthesis. In robotics, they demonstrate impressive capabilities in trajectory planning, where they learn complex policies from demonstrations [2]–[5]. These models excel at capturing multimodal behaviors and generating high-dimensional trajectories holistically, which mitigates the compounding errors common in sequential methods and improves long-term performance. However, despite their success with large datasets, diffusion models lack an intrinsic mechanism to enforce explicit constraints, such as system dynamics, obstacle avoidance, and safety margins [6]. As a result, the trajectories produced via their iterative stochastic denoising process frequently violate the physical constraints of the system [7]. To address this limitation, [8] combines a diffusion model with control barrier functions (CBFs) [9] and control Lyapunov functions (CLFs) [10] to avoid obstacles while reaching a target set during the reverse diffusion process. Although

this approach guides the planner, it only acts as a form of guidance, pushing generated trajectories toward safe and goal-directed regions. In other words, CBFs and CLFs improve the probabilistic safety of the planned trajectories, but no hard safety constraints are enforced on the final trajectory, which may still violate safety or dynamic constraints.

To enforce such hard constraints, an alternative strategy is to leverage model predictive control (MPC) [11], an optimization-based control framework that computes control actions by repeatedly solving a finite-horizon optimal control problem subject to explicit system constraints. Owing to its ability to systematically handle complex dynamics and safety requirements, MPC has been widely adopted across robotic domains [12], [13]. Recent approaches [14], [15] address this limitation by employing a hybrid framework that couples a diffusion model with MPC. This process involves generating an initial trajectory with the diffusion model, using MPC to project it onto a safe and dynamically feasible set, and then feeding the corrected trajectory back to guide the next denoising step [14]. This iterative cycle enables MPC to continually enforce safe operational boundaries. However, this hybrid strategy has a critical limitation: a poor initial trajectory from the diffusion model can prevent the MPC from converging to a feasible solution, leading to instability or failure in complex environments.

To address these challenges, we introduce D-SafeMPC, a novel framework that seamlessly integrates diffusion-based trajectory generation with MPC leveraging CBFs and CLFs. In each denoising iteration, we guide the reverse diffusion process by steering the generated trajectory samples toward safe, goal-directed regions. As aforementioned, this guidance steers the denoising process dynamically toward safe and goal-reaching trajectories without requiring retraining of the diffusion model, thereby rendering them probabilistically safer. However, this guidance alone does not enforce hard constraints such as safety or dynamic feasibility. Therefore, we refine these guided trajectories using MPC with explicit safety and dynamic constraints at each denoising step. The integration of CBFs and CLFs provides MPC with well-initialized trajectories, enabling more stable and efficient convergence to safe and feasible solutions within the allocated computational budget. The refined trajectories from MPC are then used to initialize the subsequent denoising iteration. This iterative cycle continues until the final timestep, at which point the control action from the converged trajectory is applied to the robot to advance to the next state.

The main contributions of this work are:

- We introduce a novel framework that leverages discrete-time CBFs and CLFs to guide the reverse diffusion process, steering trajectory generation toward safer, goal-

*This work was partially supported by the Horizon Europe Grant Agreement No. 101136056.

¹ E. Sayar and E. Kayacan are with the Department of Electrical Engineering and Information Technology, Paderborn University, 33098 Paderborn, Germany, {erdi.sayar, erdal.kayacan}@uni-paderborn.de.

² E. Daş is with the Department of Mechanical, Materials, and Aerospace Engineering, Illinois Institute of Technology, Chicago, IL 60616, USA, edas2@illinoistech.edu.

³ J. W. Burdick is with the Mechanical and Civil Engineering, California Institute of Technology, Pasadena, CA 91125, USA, jburdick@caltech.edu.

⁴ A. Knoll is with the School of Computation, Information, and Technology, Technical University of Munich, 80333 Munich, Germany, k@tum.de.

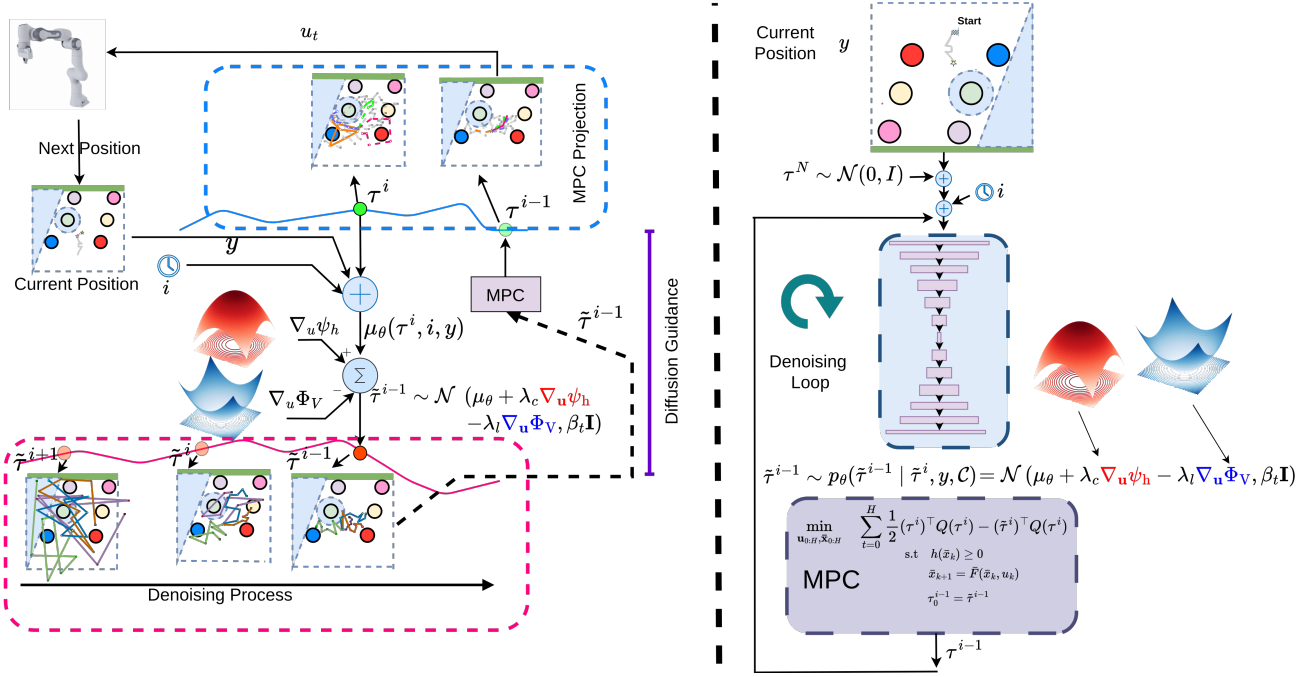


Fig. 1. Overview of the proposed diffusion-based trajectory planning framework, D-SafeMPC. The left side illustrates the evolution of trajectories during the denoising and projection process. At each denoising step, the diffusion model computes the mean $\mu_\theta(\tau^i, i, y)$ based on the projected trajectory from MPC (τ^i), the current denoising step (i) and condition information (y). This mean is then adjusted using gradients from the discrete-time CBF and CLF, denoted as $\nabla_{\mathbf{u}}\psi_h$ and $\nabla_{\mathbf{u}}\Phi_V$, respectively. The adjusted mean is then sampled to produce a trajectory $\tilde{\tau}^{i-1}$, which serves as initialization for the MPC. The MPC projects this trajectory onto a safe and dynamically feasible set τ^{i-1} , and the projected trajectory initializes the next denoising iteration. This cycle continues until the final timestep, where the control action u_t from the projected trajectory is applied to advance the robot to the next state x_t . This updated state then provides conditional input for generating the subsequent trajectory. The right side details the denoising process: the reverse diffusion loop begins with pure Gaussian noise conditioned on the current state x_t . Throughout denoising, the discrete-time CBF and CLF gradients guide the trajectory generation, steering it away from obstacles and toward the goal.

oriented regions and improving MPC initialization.

- The MPC enforces strict safety and dynamic constraints on the probabilistically safe trajectories generated via discrete-time CBF and CLF guidance, ensuring that the resulting trajectory remains within a defined safer set and adheres to the dynamic constraints.
- Empirical validation on a Franka robot manipulator in static and dynamic obstacle scenarios, demonstrating superior safety, success rates, and computational efficiency compared to state-of-the-art baselines.

II. RELATED WORK

Diffuser [2] is an offline-trained diffusion model that learns to denoise entire state-action trajectories by treating them as single-channel images. Using pre-collected trajectory datasets, the model is trained to reconstruct clean trajectories from noise through iterative temporal denoising. During inference, Diffuser produces full-length trajectories by starting from Gaussian noise and applying two mechanisms: reward-guided sampling, where a learned critic steers the denoising process toward trajectories with higher expected return, and goal-conditioned inpainting, where specified start and end states are enforced. Despite its flexibility in generating goal-directed motion, this planning approach does not guarantee safety—there is no formal mechanism to prevent unsafe states during generation. To address this safety issue, CoBL-Diffusion [8] combines a diffusion-based planner with CBFs and CLFs to enable safe navigation in dynamic environments.

In this approach, CBFs and CLFs guide the model away from obstacles and toward goals. However, this guidance acts as a *soft* preference rather than a *hard* constraint, meaning the generated trajectories might still violate safety or dynamic constraints. Similarly, other work has focused on improving physical realism by aligning the diffusion model’s sampling path with a traditional optimization trajectory, guiding the generation toward more valid solutions [16].

In contrast to these soft guidance approaches, an alternative technique is to enforce safety and goal-reaching requirements by projecting diffusion-generated trajectories onto the safe sets. The method proposed in [17] projects a noisy trajectory onto a constraint set while considering the system dynamics. Diffusion predictive control with constraints (DPCC) [14] employs a diffusion model to generate horizon-based trajectories. These trajectories serve as an initial guess for an MPC, which then projects them onto a safe and dynamically feasible set by enforcing constraints on obstacle avoidance, safe operational regions, and system dynamics. The resulting projected trajectory initializes the next denoising step in the diffusion model, creating an iterative cycle in which the MPC continually refines the trajectory to ensure safety. However, if the initial trajectory generated by the diffusion model is poor, the MPC may fail to find a feasible solution within its allocated iterations, which can pose a significant safety risk. This limitation becomes more pronounced in challenging scenarios, such as when dynamic obstacles are present or when obstacles intersect the paths of the expert trajectories

used for training. Our method addresses this by guiding the diffusion output with CBFs and CLFs, which provides a high-quality initial guess for the MPC and enables convergence.

III. BACKGROUND

Since the robot dynamics are typically continuous-time and nonlinear, we start by considering continuous-time control-affine nonlinear systems of the form:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}, \quad \mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n, \quad \mathbf{u} \in \mathcal{U} \subseteq \mathbb{R}^m, \quad (1)$$

where the drift dynamics $\mathbf{f}: \mathcal{X} \rightarrow \mathbb{R}^n$ and the actuation matrix $\mathbf{g}: \mathcal{X} \rightarrow \mathbb{R}^{n \times m}$ are locally Lipschitz continuous functions. We assume the set of admissible control inputs $\mathcal{U}$ is an m -dimensional compact convex polytope. Digital implementation of any controller introduces a sample-and-hold effect on the control:

$$\mathbf{u}(t) = \mathbf{u}_k, \quad \forall t \in [t_k, t_{k+1}), \quad (2)$$

where $\mathbf{u}_k$ is constant between consecutive sampling instants t_k and t_{k+1} with the time index $k \in \mathbb{N}_0$. We assume a uniform sampling strategy with a sampling interval T . As the system is piecewise locally Lipschitz continuous, it admits a unique trajectory. We can obtain a discrete-time nonlinear model of the continuous-time system (1) under the sample-and-hold strategy (2) as

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \int_{t_k}^{t_{k+1}} (\mathbf{f}(\mathbf{x}(\tau)) + \mathbf{g}(\mathbf{x}(\tau))\mathbf{u}_k) d\tau \triangleq \mathbf{F}(\mathbf{x}_k, \mathbf{u}_k), \quad (3)$$

where $\mathbf{F}: \mathcal{X} \times \mathcal{U} \rightarrow \mathcal{X}$ is the *exact state discrete map*. If we use a one-step Euler discretization to approximate this map, we have $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{u}_k) = \mathbf{x}_k + (\mathbf{f}(\mathbf{x}_k) + \mathbf{g}(\mathbf{x}_k)\mathbf{u}_k)T + \mathcal{O}(T^2)$. Throughout the paper, we denote $\mathbf{F}$ as the discrete-time model used for control and analysis. When necessary, $\mathbf{F}$ refers to the Euler approximation of the exact flow map.

A. Discrete-Time CBFs and CLFs

We consider dynamic obstacles in our safe control scenarios, and our control framework is discrete-time. We first define discrete-time control barrier functions (CBFs) [18], [19], which extend continuous-time CBFs [9] to certify safety at sampling instants.

We characterize safety through the notion of forward invariance. Let the time-invariant set $\mathcal{C}$ be the 0-superlevel set of a continuously differentiable function $h: \mathcal{X} \rightarrow \mathbb{R}$:

$$\mathcal{C} \triangleq \{\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n : h(\mathbf{x}) \geq 0\}. \quad (4)$$

This set is said to be *forward invariant* if for the discrete-time system (3), for any initial condition $\mathbf{x}_0 \in \mathcal{C}$, implies that $\mathbf{x}_k \in \mathcal{C}$, for all $k \in \mathbb{N}_0$. The closed-loop system $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{k}(\mathbf{x}_k))$, with a feedback controller $\mathbf{k}: \mathcal{X} \rightarrow \mathcal{U}$, $\mathbf{u} = \mathbf{k}(\mathbf{x})$, is *safe* with respect to the *safe set* $\mathcal{C}$, if $\mathcal{C}$ is forward invariant. To design controllers that guarantee such forward invariance properties, discrete-time control barrier functions (CBFs) provide a systematic framework.

Definition 1 (Discrete-Time Control Barrier Function). Let $\mathcal{C} \subseteq \mathcal{X}$ be the 0-superlevel set of a continuously differentiable function $h: \mathcal{X} \rightarrow \mathbb{R}$. The function h is a *discrete-time control barrier function* for system (3) with respect to $\mathcal{C}$ if there

exists a class- $\mathcal{K}$ function* $\alpha \in \mathcal{K}$ satisfying $\alpha(r) < r, \forall r > 0$ such that $\forall \mathbf{x}_k \in \mathcal{C}$:

$$\sup_{\mathbf{u}_k \in \mathcal{U}} [h(\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k)) - h(\mathbf{x}_k)] > -\alpha(h(\mathbf{x}_k)).$$

We can derive formal safety guarantees using Definition 1, with the help of the following theorem:

Theorem 1. If h is a discrete-time CBF for (3) on $\mathcal{C}$, then any feedback controller $\mathbf{k}: \mathcal{X} \rightarrow \mathcal{U}$, $\mathbf{u}_k = \mathbf{k}(\mathbf{x}_k)$ satisfying

$$h(\mathbf{F}(\mathbf{x}_k, \mathbf{k}(\mathbf{x}_k))) - h(\mathbf{x}_k) \geq -\alpha(h(\mathbf{x}_k)) \quad (5)$$

for all $\mathbf{x}_k \in \mathcal{C}$ renders the set $\mathcal{C}$ forward invariant.

Similar to CBFs, CLFs encode stability objectives by requiring a Lyapunov candidate to decrease along closed-loop trajectories. Therefore, CLFs are useful for representing closed-loop objectives, such as stabilizing to a desired configuration or, through an appropriate choice of CLFs, converging to a target set. We specifically consider discrete-time exponential control Lyapunov functions (CLFs):

Definition 2 (Discrete-Time Control Lyapunov Function). A continuously differentiable function $V(x)$, $V: \mathcal{G} \subseteq \mathcal{X} \rightarrow \mathbb{R}_{\geq 0}$, is a *discrete-time control Lyapunov function* for system (3) if there exists a constant $\sigma \in (0, 1)$ such that for all $\mathbf{x}_k \in \mathcal{G}$:

$$\inf_{\mathbf{u}_k \in \mathcal{U}} [V(\mathbf{F}(\mathbf{x}_k, \mathbf{u}_k)) - V(\mathbf{x}_k)] \leq -\sigma V(\mathbf{x}_k).$$

B. Diffusion Models

Diffusion models [1] express a probability distribution $p(\tau_0)$ through latent variables in the form $p_\theta(\tau_0) \triangleq \int p_\theta(\tau_{0:N}) d\tau_{1:N}$, where $\tau_1, \dots, \tau_N$ are latent variables of the same dimensionality as the data $\tau_0 \sim p(\tau_0)$.

These models are characterized by a forward and a reverse diffusion process. The forward diffusion process is the fixed approximate posterior $q(\tau_{1:N} | \tau_0)$, given by a Markov chain that, starting from $\tau_0 \sim q(\tau_0)$, perturbs the input data by gradually adding Gaussian noise in N steps with a predefined variance schedule $\beta_1, \dots, \beta_N$ following the cosine noise schedule [20]. This process is defined as

$$q(\tau_{1:N} | \tau_0) \triangleq \prod_{i=1}^N q(\tau_i | \tau_{i-1}), \quad (6)$$

$$q(\tau_i | \tau_{i-1}) \triangleq \mathcal{N}(\tau_i; \sqrt{1 - \beta_i} \tau_{i-1}, \beta_i \mathbf{I}).$$

The reverse diffusion process aims to recover the original input data from the noisy (diffused) data. A deep neural network with parameters θ is trained to progressively reverse the diffusion process step by step and approximates the joint distribution $p_\theta(\tau_{0:N})$. This process is modelled as [1]

$$p_\theta(\tau_{0:N}) \triangleq p(\tau_N) \prod_{i=1}^N p_\theta(\tau_{i-1} | \tau_i), \quad (7)$$

$$p_\theta(\tau_{i-1} | \tau_i) \triangleq \mathcal{N}(\tau_{i-1}; \boldsymbol{\mu}_\theta(\tau_i, i), \boldsymbol{\Sigma}_\theta(\tau_i, i)),$$

where $p(\tau_N) = \mathcal{N}(\tau_N; \mathbf{0}, \mathbf{I})$, $\boldsymbol{\mu}_\theta$ and $\boldsymbol{\Sigma}_\theta$ are the outputs of the neural network. Following [1], we set $\boldsymbol{\Sigma}_\theta = \sigma_i^2 \mathbf{I} = \beta_i \mathbf{I}$,

*A continuous function $\alpha: [0, a) \rightarrow \mathbb{R}_{\geq 0}$, where $a > 0$, belongs to class- $\mathcal{K}$ ($\alpha \in \mathcal{K}$) if it is strictly monotonically increasing and $\alpha(0) = 0$.

which is not learned but follows the cosine schedule, and the mean is defined as

$$\mu_\theta(\tau_i, i) = \frac{1}{\sqrt{\alpha_i}} \left(\tau_i - \frac{\beta_i}{\sqrt{1 - \bar{\alpha}_i}} \epsilon_\theta(\tau_i, i) \right), \quad (8)$$

where $\alpha_i = 1 - \beta_i$ and $\bar{\alpha}_i = \prod_{s=1}^N \alpha_s$ is the cumulative product of the α terms. The model is optimized by maximizing the evidence lower bound (ELBO) [21], $\mathbb{E}_q \left[\ln \frac{p_\theta(\tau_{0:N})}{q(\tau_{1:N}|\tau_0)} \right]$. The training objective can be simplified from the ELBO to a loss function [1]:

$$\mathcal{L}_d(\theta) = \mathbb{E}_{i, \epsilon, \tau_0} \left[\left\| \epsilon - \epsilon_\theta \left(\underbrace{\sqrt{\bar{\alpha}_i} \tau_0 + \sqrt{1 - \bar{\alpha}_i} \epsilon}_{\tau_i}, i \right) \right\|^2 \right], \quad (9)$$

here, ϵ_θ is a neural network that learns to predict the noise. Once trained, sampling data from Gaussian noise $\tau_N \sim p(\tau_N)$ and running it through the reverse diffusion process from $i=N$ to $i=0$ yields an approximation of the original data distribution.

IV. TRAJECTORY PLANNING WITH CONDITIONAL DIFFUSION MODELS

As planning and diffusion processes operate on distinct temporal values, we denote the diffusion iteration index with a superscript $i \in \{0, \dots, N\}$ and the planning trajectory index with a subscript $t \in \{0, \dots, T\}$. Following this convention, $\tau^i = (\mathbf{x}_0^i, \mathbf{u}_0^i, \dots, \mathbf{x}_H^i)$ represents the entire trajectory at diffusion step i with a horizon H . However, an important distinction is that we denote the output of the guided diffusion model with $\tilde{\tau}^i$ and the output of the MPC projection with τ^i .

We consider a robotic system operating in an environment with N_J static or dynamic obstacles. The system is governed by the discrete-time, nonlinear dynamics given in (3). The state and control input of the j -th obstacle at timestep t is denoted by $\mathbf{x}_{j,t}$ and $\mathbf{u}_{j,t}$, for $j = 1, \dots, N_J$.

Our objective is to control the robot from an initial state $\mathbf{x}_0$ to a goal $g \in \mathcal{G}$ within a maximum of T time steps. Goal achievement is indicated by a binary function $\phi: \mathcal{X} \times \mathcal{G} \rightarrow \{0, 1\}$, where $\phi(\mathbf{x}_T, g) = 1$ signals success. In our examples, we use the D3IL dataset [22], which contains teleoperated demonstrations with a 7-DoF Franka Emika Panda robot in MuJoCo [23]. This dataset, $\mathcal{D} = \{^{(m)}\tau_e\}_{m=1}^M$, consists of M successful expert trajectories of the form $^{(m)}\tau_e = (^{(m)}\mathbf{x}_0, ^{(m)}\mathbf{u}_0, \dots, ^{(m)}\mathbf{x}_T)$, each associated with a goal $^{(m)}g$ such that $\phi(^{(m)}\mathbf{x}_T, ^{(m)}g) = 1$. The expert is modeled as an unknown stochastic policy π_e , where actions are sampled as $\mathbf{u}_t \sim \pi_e(\cdot | \mathbf{x}_t, g)$. We aim to learn a goal-conditioned stochastic control policy π from the expert dataset $\mathcal{D}$ using a conditional diffusion model. As recent work [2] has demonstrated that conditional diffusion models can effectively learn complex robotic control policies from offline data, we adopt this method. Our diffusion model is trained to approximate the conditional distribution of trajectories $p_\theta(\tau | y)$ where y is the conditioning information, such as the current state $\mathbf{x}_t$ and a desired goal g .

During the training process, the diffusion model learns from offline trajectory data collected in obstacle-free environments, implicitly encoding dynamic constraints from the demonstration data. However, during inference, the environment may

contain obstacles and unsafe regions not present in the training data. While the diffusion model can generate trajectories with horizon H conditioned on y that respect the learned dynamics, the stochastic nature of the reverse diffusion process may produce trajectories that violate safety constraints or dynamic feasibility. To address these safety concerns, we define a safe set, denoted by $\mathcal{C}$, using a CBF, $h(\mathbf{x})$. This set consists of all the states $\mathbf{x}$ for which the CBF is non-negative, as formally defined by the set in (4).

We can incorporate these constraints into the diffusion architecture and train the model in environments with dynamic obstacles. However, learning the distribution $p_\theta(\tilde{\tau}^{i-1} | \tilde{\tau}^i, y, \mathcal{C})$ requires extensive offline demonstrations that satisfy the safety constraints. Furthermore, since $\mathcal{C}$ is task-specific, the trained model might fail to generalize during inference for unseen scenarios.

To overcome these challenges, we propose a more flexible approach: incorporating the safety constraints as soft constraints during the denoising process. We achieve this using classifier guidance [24], which dynamically steers trajectory generation toward safe regions without requiring model retraining. Specifically, we model the likelihood of a trajectory $\tilde{\tau}^{i-1}$ belonging to the safe set $\mathcal{C}$ using a Gibbs-Boltzmann distribution:

$$p(\tilde{\tau}^{i-1} \in \mathcal{C}) \propto \exp \left(\frac{h(\tilde{\tau}^{i-1})}{\mathcal{T}} \right), \quad (10)$$

where $\mathcal{T}$ is a temperature parameter, and trajectories with higher (safer) h values are assigned a higher probability, effectively directing the reverse diffusion process toward desired outcomes.

Our objective is thus to sample from the posterior $p(\tau^0 | y, \mathcal{C})$, expressed as

$$p(\tau^0 | y, \mathcal{C}) = \int p(\tau^N | y, \mathcal{C}) \prod_{i=1}^N p(\tau^{i-1} | \tau^i, i, y, \mathcal{C}) d\tau_{1:N} \quad (11)$$

with $p(\tau^N | y, \mathcal{C}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$. Sampling from this posterior involves iteratively drawing from the safety-conditioned distribution via Bayes' rule:

$$p(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y, \mathcal{C}) = Z \cdot p(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y) \cdot p(\mathcal{C} | \tilde{\tau}^{i-1}), \quad (12)$$

where Z is a normalizing constant. Because Z is typically intractable to compute, it is not possible to sample from this distribution directly [24]. Therefore, the safety-conditioned distribution $p(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y, \mathcal{C})$ has no closed form for sampling. Taking the logarithm of this safety-conditioned posterior yields

$$\log p(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y, \mathcal{C}) \approx \log p(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y) + \log p(\mathcal{C} | \tilde{\tau}^{i-1}), \quad (13)$$

where $p(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y) \approx p_\theta(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y)$ denotes the learned denoising model. This decomposition separates the learned diffusion denoising from the safety guidance, and the learned denoising model is given by

$$\begin{aligned} \log p_\theta(\tilde{\tau}^{i-1} | \tilde{\tau}^i, i, y) &= \log \mathcal{N}(\mu_\theta, \beta_i \mathbf{I}) \\ &= -\frac{1}{2} (\tilde{\tau}^{i-1} - \mu_\theta)^\top \left(\frac{\mathbf{I}}{\beta_i} \right) (\tilde{\tau}^{i-1} - \mu_\theta) + c_1, \end{aligned} \quad (14)$$

where c_1 is a constant and $\mu_\theta = \mu_\theta(\tilde{\tau}^i, i, y)$ for brevity. We approximate $\log p(\mathcal{C} \mid \tilde{\tau}^{i-1})$ using a first-order Taylor expansion around the mean $\tilde{\tau}^{i-1} = \mu_\theta$:

$$\begin{aligned} \log p(\mathcal{C} \mid \tilde{\tau}^{i-1}) &\approx \log p(\mathcal{C} \mid \tilde{\tau}^{i-1}) \Big|_{\tilde{\tau}^{i-1}=\mu_\theta} + (\tilde{\tau}^{i-1} - \mu_\theta) \cdot g \\ &\approx c_2 + (\tilde{\tau}^{i-1} - \mu_\theta) \cdot g, \end{aligned} \quad (15)$$

where c_2 is the constant value at the mean, and $g = \nabla_{\tilde{\tau}^{i-1}} \log p(\mathcal{C} \mid \tilde{\tau}^{i-1}) \Big|_{\tilde{\tau}^{i-1}=\mu_\theta}$ is the gradient of the safety log-probability evaluated at μ_θ . This linear approximation is valid locally and enables gradient-based guidance. Combining the approximations, the conditional log-probability becomes:

$$\begin{aligned} \log p_\theta(\tilde{\tau}^{i-1} \mid \tilde{\tau}^i, y, \mathcal{C}) \\ \approx -\frac{1}{2\beta_i} \left\| \tilde{\tau}^{i-1} - \left(\mu_\theta + \frac{\beta_i}{\mathcal{T}} \nabla_{\tilde{\tau}^{i-1}} h(\tilde{\tau}^{i-1}) \Big|_{\tilde{\tau}^{i-1}=\mu_\theta} \right) \right\|^2 + c_3 \\ \approx -\frac{1}{2\beta_i} \left\| \tilde{\tau}^{i-1} - (\mu_\theta + \beta_i g) \right\|^2 + c_3, \end{aligned} \quad (16)$$

where c_3 represents all constant terms. This approximation in (16) reveals that the conditional distribution remains Gaussian, with its mean shifted in the direction of the safety constraint's gradient. Ultimately, since we rely on control actions to govern the robot's behavior, we apply this guidance by computing the gradients of the CBF and CLF conditions (as detailed in Section IV-A) with respect to the control input, rather than the entire trajectory. CBFs, as detailed in Section III-A, facilitate guiding the reverse diffusion process to produce trajectories that avoid collisions with static or dynamic obstacles. Similarly, CLFs can be utilized to steer the reverse diffusion process toward the goal.

A. CBF and CLF Guidance Design

Since we consider both static and dynamic obstacles, their dynamics affect the CBF-based safety and CLF-based stability conditions. Therefore, we augment the nonlinear system model (1) with the autonomous dynamics of these obstacles:

$$\dot{\mathbf{x}}_j = \mathbf{f}_j(\mathbf{x}_j), \quad j = 1, \dots, N_J, \quad (17)$$

where $\mathbf{x}_j \in \mathbb{R}^{n_j}$. We assume the future obstacle states $\mathbf{x}_{j,t+1}, \dots, \mathbf{x}_{j,t+H}$ over the planning horizon H are known to the planner. Then, we define an augmented system dynamics formed by (1) and (17) as

$$\underbrace{\begin{bmatrix} \dot{\mathbf{x}} \\ \dot{\mathbf{x}}_1 \\ \vdots \\ \dot{\mathbf{x}}_j \\ \vdots \\ \dot{\mathbf{x}}_{N_J} \end{bmatrix}}_{\triangleq \dot{\bar{\mathbf{x}}}} = \underbrace{\begin{bmatrix} \mathbf{f}(\mathbf{x}) \\ \mathbf{f}_1(\mathbf{x}_1) \\ \vdots \\ \mathbf{f}_j(\mathbf{x}_j) \\ \vdots \\ \mathbf{f}_{N_J}(\mathbf{x}_{N_J}) \end{bmatrix}}_{\triangleq \bar{\mathbf{f}}(\bar{\mathbf{x}})} + \underbrace{\begin{bmatrix} \mathbf{g}(\mathbf{x}) \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \\ \vdots \\ \mathbf{0} \end{bmatrix}}_{\triangleq \bar{\mathbf{g}}(\bar{\mathbf{x}})} \mathbf{u}, \quad (18)$$

where $\bar{\mathbf{x}} \in \mathbb{R}^{n+n_J \times N_J}$ is the augmented state. We represent discrete-time dynamics as $\bar{\mathbf{F}}(\bar{\mathbf{x}}_k, \mathbf{u}_k)$.

Next, we consider the collision avoidance between the robot and the j -th obstacle, which has a radius r_j :

$$h_j(\bar{\mathbf{x}}) = \|\mathbf{x} - \mathbf{x}_j\|^2 - r_j^2. \quad (19)$$

Our goal is to satisfy all constraints simultaneously; therefore,

we define a unified safety function using the min function:

$$h(\bar{\mathbf{x}}) = \min_{j \in \{1, \dots, N_J\}} [h_j(\bar{\mathbf{x}})], \quad (20)$$

which is non-smooth. To construct a smooth function representing the min function, we can employ a smooth under-approximation given by (with $\lambda \geq 0$):

$$h(\bar{\mathbf{x}}) = -\frac{1}{\lambda} \ln \left(\sum_{j=1}^{N_J} e^{-\lambda h_j(\bar{\mathbf{x}})} \right). \quad (21)$$

From Theorem 1, we have a formal safety guarantee with respect to h if the discrete-time CBF condition:

$$0 \leq h(\bar{\mathbf{F}}(\bar{\mathbf{x}}_k, \mathbf{u}_k)) - h(\bar{\mathbf{x}}_k) + \alpha(h(\bar{\mathbf{x}}_k)) \triangleq \psi_h(\bar{\mathbf{x}}_k, \mathbf{u}_k) \quad (22)$$

is satisfied for all sampled $\bar{\mathbf{x}}_k$ via control input $\mathbf{u}_k$.

Therefore, the collision avoidance safety (21) is guaranteed if the safety condition, ψ_h , remains positive for every control action throughout the entire sequence. To actively enforce this using the guidance mechanism shown in (16), we utilize the gradient of the safety condition with respect to the control input:

$$\nabla_{\mathbf{u}} \psi_h(\bar{\mathbf{x}}, \mathbf{u}) = \left(\frac{\partial \bar{\mathbf{F}}(\bar{\mathbf{x}}, \mathbf{u})}{\partial \mathbf{u}} \right)^\top \nabla_{\bar{\mathbf{x}}} h(\bar{\mathbf{F}}(\bar{\mathbf{x}}, \mathbf{u})), \quad (23)$$

which points in the direction of steepest ascent of ψ_h with respect to $\mathbf{u}$, indicating how to modify the control input to improve satisfaction of the discrete-time CBF condition [8]. Under the Euler discretization, $\frac{\partial \bar{\mathbf{F}}}{\partial \mathbf{u}} = T \bar{\mathbf{g}}(\bar{\mathbf{x}})$ depends only on $\bar{\mathbf{x}}$. Nevertheless, $\nabla_{\mathbf{u}} \psi_h(\bar{\mathbf{x}}, \mathbf{u})$ generally depends on $\mathbf{u}$ through the evaluation of $\nabla_{\bar{\mathbf{x}}} h$ at $\bar{\mathbf{F}}(\bar{\mathbf{x}}, \mathbf{u})$. By integrating this gradient as a guidance signal within each denoising step, evaluated at the current denoising iterate, we steer the diffusion model toward synthesizing trajectories directed toward the safe set.

Similar to how a CBF enforces safety by repelling the denoising steps from unsafe regions, a CLF can enforce stability by attracting the denoising steps toward a desired goal state. To guide the reverse diffusion process toward the goal state g , we define a CLF:

$$V(\bar{\mathbf{x}}) = \|\bar{\mathbf{x}} - g\|^2. \quad (24)$$

The condition ensures exponential stability of g is given by

$$0 \geq V(\bar{\mathbf{F}}(\bar{\mathbf{x}}_k, \mathbf{u}_k)) - V(\bar{\mathbf{x}}_k) + \sigma V(\bar{\mathbf{x}}_k) \triangleq \Phi_V(\bar{\mathbf{x}}_k, \mathbf{u}_k), \quad (25)$$

The gradient of Φ_V with respect to $\mathbf{u}$:

$$\nabla_{\mathbf{u}} \Phi_V(\bar{\mathbf{x}}, \mathbf{u}) = \left(\frac{\partial \bar{\mathbf{F}}(\bar{\mathbf{x}}, \mathbf{u})}{\partial \mathbf{u}} \right)^\top \nabla_{\bar{\mathbf{x}}} V(\bar{\mathbf{F}}(\bar{\mathbf{x}}, \mathbf{u})), \quad (26)$$

which indicates the direction in which $\mathbf{u}$ should be selected to most effectively decrease the CLF value, thereby enabling convergence to the goal state g .

By integrating gradients from both CBFs and CLFs into the denoising process as

$$p_\theta(\tilde{\tau}^{i-1} \mid \tilde{\tau}^i, y) = \mathcal{N}(\mu_\theta + \lambda_c \nabla_{\mathbf{u}} \psi_h - \lambda_l \nabla_{\mathbf{u}} \Phi_V, \beta_i \mathbf{I}), \quad (27)$$

the diffusion model generates trajectories that avoid obstacles while advancing toward the goal, thereby reducing the risk of stagnation in safe yet suboptimal regions. Nevertheless, conflicts may arise when the CBF and CLF gradients oppose one another. To address this, we employ an adaptive weight

$\lambda_c \geq 0$ for the CBF constraint, which increases proportionally as the distance to obstacles decreases, while maintaining a fixed weight $\lambda_l \geq 0$ for the CLF constraint. Note that we set both $\lambda_c, \lambda_l = 0$ during the first few denoising steps, as the initial noisy trajectories cannot be expected to satisfy safety constraints.

Although CBFs and CLFs guide the generated trajectories by steering them away from obstacles and toward desired goals, respectively, this guidance serves as a soft directive rather than a hard constraint, steering the model toward approximately feasible regions. As a result, safety violations may still arise in the generated trajectories. To mitigate this issue, we utilize an MPC as a corrective refinement to enhance the trajectories, guaranteeing compliance with both dynamic and safety constraints. We can formulate the MPC as

$$\begin{aligned} \min_{\mathbf{u}_{0:H}, \tilde{\mathbf{x}}_{0:H}} \quad & \sum_{t=0}^H \frac{1}{2} (\tau^t)^\top Q(\tau^t) - (\tilde{\tau}^t)^\top Q(\tau^t) \\ \text{s.t.} \quad & h(\tilde{\mathbf{x}}_k) \geq 0 \\ & \tilde{\mathbf{x}}_{k+1} = \bar{\mathbf{F}}(\tilde{\mathbf{x}}_k, \mathbf{u}_k) \\ & \tau_0^{i-1} = \tilde{\tau}^{i-1} \end{aligned} \quad (28)$$

where $\tau^i \triangleq (\tilde{\mathbf{x}}_0^i, \mathbf{u}_0^i, \dots, \tilde{\mathbf{x}}_H^i)$ and $\tilde{\tau}^{i-1}$ is the warm-start trajectory from the CBF- and CLF-guided diffusion process, providing a probabilistically safe initialization for the MPC. This initialization enables the MPC to converge more rapidly to feasible, goal-oriented solutions compared to unguided cases, where it may fail to identify valid trajectories within allocated iteration limits or become trapped in safe yet suboptimal regions. In the MPC objective function given in (28), the first term, $\frac{1}{2} (\tau^i)^\top Q(\tau^i)$, serves as a regularization term that prevents the optimization variables from growing excessively large. The second term, $-(\tilde{\tau}^i)^\top Q(\tau^i)$, encourages the optimization variables to remain close to the output of the guided diffusion model, ensuring that the resulting trajectories do not deviate significantly from the distribution learned by the diffusion neural network. Similar to the approach in DPCC [14], our framework employs an iterative projection scheme that tightly integrates MPC into the reverse diffusion process. At each denoising iteration, the MPC projects the probabilistically safe trajectory $\tilde{\tau}^{i-1}$ onto a safe and dynamically feasible set. This is achieved by using $\tilde{\tau}^{i-1}$ to both initialize the MPC's solver (as a warm start) and penalize deviations from this trajectory in the cost function. The resulting corrected trajectory then initializes the subsequent denoising step. In this refinement loop, MPC serves as a dedicated projection operator, rigorously enforcing constraints throughout the generation process. Although only the final denoised trajectory is executed, enforcing feasibility during intermediate steps is important due to the sequential (Markovian) nature of reverse diffusion: each iterate initializes the subsequent denoising step and, in our method, also warm-starts the MPC projection. Consequently, a highly unsafe or dynamically infeasible intermediate trajectory can propagate errors forward by biasing subsequent denoising updates or by preventing the MPC from converging within a fixed

*Since the data contain more noise in the early denoising iterations, guidance and MPC projection are excluded for the first few steps.

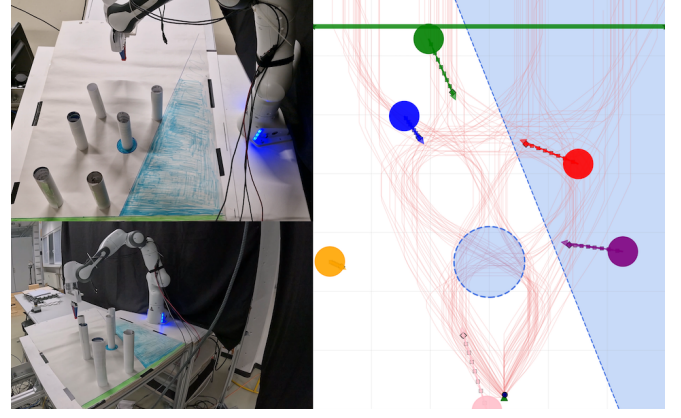


Fig. 2. Experimental setups for static (left) and dynamic (right) obstacle avoidance scenarios. The left panel shows the physical setup for our Sim2Real experiments, featuring static obstacles (white cylinders), a designated safe area (blue), and a goal line (green). The right panel displays the simulation environment with dynamic obstacles (colored circles), whose future paths are indicated by arrows. The faint red lines represent the numerous expert trajectories trained by our framework to safely navigate the environment, while the light blue area indicates a safety constraint.

Algorithm 1 D-SafeMPC with CBF/CLF Guidance

```

1: Input: Guidance weights  $\lambda_c, \lambda_l$ 
2: Initialize trajectory  $\tau^N \sim \mathcal{N}(0, I)$ 
3: for  $i = N \dots 1$  do
4:    $z \sim \mathcal{N}(0, I)$  if  $i > 1$ , else  $z = 0$ 
5:    $\mu_\theta = \frac{1}{\sqrt{\alpha_i}} \left( \tau^i - \frac{\beta_i}{\sqrt{1-\alpha_i}} \epsilon_\theta(\tau^i, i, y) \right)$ 
6:   Denoise with CBF/CLF guidance
7:    $\hat{\mu} \leftarrow \mu_\theta + \lambda_c \nabla_{\mathbf{u}} \psi_h - \lambda_l \nabla_{\mathbf{u}} \Phi_V$ 
8:    $\tilde{\tau}^{i-1} \leftarrow \hat{\mu} + z \sigma_i$   $\triangleright$  Sample using reparameterization trick
9:   if  $i \leq N/2$  then
10:     $\tau^{i-1} \leftarrow \text{MPC-Project}(\tilde{\tau}^{i-1})$   $\triangleright$  Eq.(28)
11:   else
12:     $\tau^{i-1} \leftarrow \tilde{\tau}^{i-1}$ 
13:   end if
14: end for
15: return  $\tau^0$ 

```

computational budget, ultimately degrading the final plan. Following DPCC [14], we therefore apply the projection operator during denoising, but only after an initial burn-in phase (once the sample leaves the pure-noise regime); from that point onward, the projected feasible iterate is fed back to the diffusion model to keep the remaining denoising iterations within the basin of attraction of the constrained optimizer. The overall framework is visualized in Fig. 1.

Algorithm 1 outlines the D-SafeMPC framework. The framework begins by initializing a trajectory with pure Gaussian noise (line 2). In iterative mode (lines 3–14), it employs a step-by-step reverse diffusion process from diffusion step N down to 1. In each iteration, the following steps occur:

- **Guidance:** The predicted mean of the reverse process, μ , (line 5) is adjusted using gradients from CBFs and CLFs (line 7). This critical step steers the trajectory generation toward safer, goal-oriented regions while avoiding obstacles.

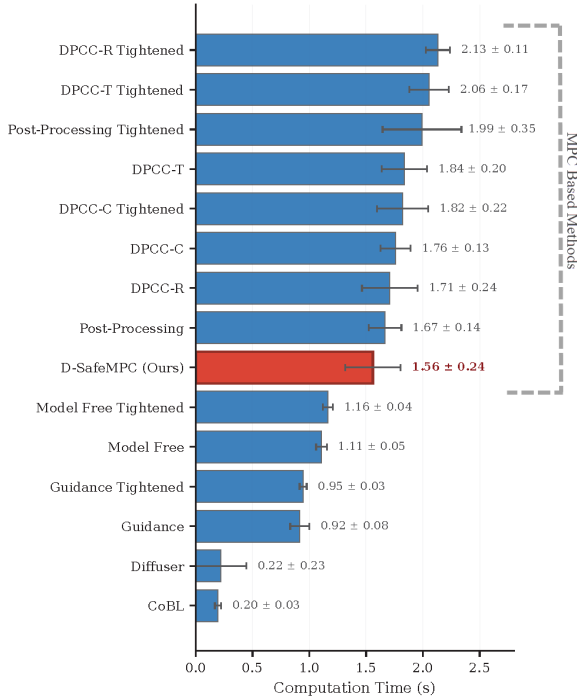


Fig. 3. A comparison of the average time (in seconds) each planner requires to generate a control action in the static obstacle environment.

- Sampling: A new, less noisy trajectory, $\tilde{\tau}^{i-1}$, is sampled from a distribution centered at the guided mean, $\hat{\mu}$ (line 8).
- Projection: Once the trajectory leaves the high-noise regime ($i \leq N/2$)[†], an MPC projects it onto a safe and dynamically feasible set (line 10) as expressed in (28). The resulting corrected trajectory, τ^{i-1} , is then used to initialize the subsequent denoising step, ensuring that safety and system constraints are enforced throughout the entire generation process.

V. EXPERIMENTS

In our simulation, we create realistic dynamic environments for robotic path planning by modeling obstacle motion with parametric trajectories. Obstacles are categorized as either static, with fixed positions, or dynamic, following circular orbits with a defined angular velocity.

For each dynamic obstacle, we generate an ideal reference trajectory in $\mathbb{R}^2$, denoted by $p_{\text{ref}}(t)$. The position reference at time t is defined as $x_{\text{ref}}(t) = x_c + r \cos(\omega t + \phi)$ and $y_{\text{ref}}(t) = y_c + r \sin(\omega t + \phi)$ where (x_c, y_c) is the center of the circular path, r is the radius, ω is the angular velocity, and ϕ is the initial phase offset. For static obstacles, the reference trajectory is a fixed point, which is achieved by setting the radius and angular velocity to zero ($r = 0, \omega = 0$).

To create a diverse set of scenarios, the parameters for each obstacle’s trajectory—radius, angular velocity, and phase offset—are sampled from uniform distributions within specified bounds. The actual obstacle trajectories are simulated using a proportional-derivative (PD) controller to track the

reference path. The system’s dynamics are modeled as a velocity-controlled point mass, with the control input determined by the PD control law. The simulation uses forward Euler integration to update the obstacle’s position at each timestep. In all experiments, we use $N = 20$ diffusion steps; consequently, the burn-in threshold in Algorithm 1 corresponds to $i \leq N/2 = 10$.

We evaluate all planners across 10 random seeds using two metrics (Table I)[‡]. The *Goal Reach Rate* measures the proportion of such collision-free goal-reaching trials. The *Safety-Compliant Goal Reached Rate* additionally requires satisfaction of all safety constraints throughout the trajectory.

Our proposed framework, D-SafeMPC, demonstrates consistently superior performance over multiple baselines across both static and dynamic environments. In the static obstacle scenario, while CoBL achieves marginally higher success rates and DPCC-C attains better safe-success rates, our method remains highly competitive, with D-SafeMPC yielding 80% success and safe-success. Notably, D-SafeMPC excels in dynamic obstacle scenarios, where it achieves the highest or near-highest success and safe-success rates, underscoring its robustness in enforcing safety constraints under challenging conditions. In addition to task performance, we evaluated the computational efficiency of each planner. Figure 3 illustrates the average computation time per decision step for each method in the static environment. Our proposed framework, D-SafeMPC, achieves a favorable balance between its high safe-success rates and computational overhead. Notably, D-SafeMPC is computationally more efficient than other MPC-based models, such as the DPCC variants and Post-Processing. We attribute this improvement to the CBF and CLF guidance, which provides the MPC with better-initialized trajectories, leading to faster convergence. While methods without an MPC projection step, such as Diffuser, are significantly faster, their speed comes at the cost of lower safety and reliability, as reflected in their poor safe-success rates.

A. Sim-to-Real Experiments

We have validated our framework with a sim-to-real experiment, using the physical (static obstacles) and simulated (dynamic obstacles) setups shown in Fig. 2. The diffusion model, trained entirely in simulation, has been deployed on a physical Franka robot manipulator. Control and state feedback have been handled using the Franky library[§]. During this sim-to-real experiment, we have scaled the dynamic parameters of the real robot by setting the relative dynamics factor to 0.05 in order to minimize controller errors and ensure precise positioning, which is crucial for successfully avoiding obstacles and unsafe regions. In the experiment, the robot has executed a trajectory guided by CBFs and CLFs, with final optimization performed by the MPC. We have demonstrated the real-world performance of our proposed D-SafeMPC and DPCC baseline in a sim-to-real experiment.

VI. CONCLUSION

In this work, we introduce D-SafeMPC, a novel framework that addresses the critical challenge of enforcing safety and

[†]To ensure a fair comparison, we adopt the same burn-in strategy and hyperparameters as the DPCC baseline [14].

[‡]Each planner is tested with two variants per seed, totaling 20 evaluations.

[§]<https://github.com/TimSchneider42/franky>

TABLE I. Planning performance in static and dynamic environments, showing average success and safe-success rates.

Planner	Static Env		Dynamic Env 1		Dynamic Env 2		Dynamic Env 3	
	Safety-Compliant Goal Reached Rate	Goal Reached Rate	Safety-Compliant Goal Reached Rate	Goal Reached Rate	Safety-Compliant Goal Reached Rate	Goal Reached Rate	Safety-Compliant Goal Reached Rate	Goal Reached Rate
D-SafeMPC (ours)	0.800 ± 0.332	0.800 ± 0.332	1.000	1.000	0.875 ± 0.268	0.875 ± 0.268	0.925 ± 0.179	0.925 ± 0.179
CoBL	0.050 ± 0.150	1.000	0.100 ± 0.300	0.850 ± 0.320	0.775 ± 0.370	0.925 ± 0.238	0.800 ± 0.332	0.850 ± 0.320
Diffuser	0.050 ± 0.150	0.650 ± 0.450	0.200 ± 0.332	0.400 ± 0.436	0.175 ± 0.327	0.325 ± 0.426	0.325 ± 0.426	0.450 ± 0.415
DPCC-C	0.900 ± 0.300	0.900 ± 0.300	0.750 ± 0.335	0.850 ± 0.229	0.450 ± 0.415	0.600 ± 0.339	0.425 ± 0.327	0.425 ± 0.327
DPCC-C Tightened	0.700 ± 0.400	0.700 ± 0.400	0.700 ± 0.332	0.750 ± 0.250	0.575 ± 0.396	0.575 ± 0.396	0.650 ± 0.391	0.650 ± 0.391
DPCC-R	0.350 ± 0.391	0.500 ± 0.387	0.450 ± 0.350	0.650 ± 0.391	0.275 ± 0.295	0.325 ± 0.327	0.675 ± 0.286	0.825 ± 0.238
DPCC-R Tightened	0.700 ± 0.332	0.700 ± 0.332	0.800 ± 0.332	0.900 ± 0.300	0.600 ± 0.339	0.600 ± 0.339	0.750 ± 0.250	0.750 ± 0.250
DPCC-T	0.800 ± 0.332	0.950 ± 0.150	0.350 ± 0.320	0.500 ± 0.316	0.050 ± 0.150	0.050 ± 0.150	0.275 ± 0.334	0.300 ± 0.367
DPCC-T Tightened	0.850 ± 0.320	0.850 ± 0.320	0.450 ± 0.350	0.450 ± 0.350	0.125 ± 0.217	0.125 ± 0.217	0.350 ± 0.421	0.350 ± 0.421
Guidance	0.400 ± 0.490	0.400 ± 0.490	0.400 ± 0.490	0.400 ± 0.490	0.525 ± 0.487	0.525 ± 0.487	0.525 ± 0.487	0.525 ± 0.487
Guidance Tightened	0.400 ± 0.490	0.400 ± 0.490	0.400 ± 0.490	0.400 ± 0.490	0.525 ± 0.487	0.525 ± 0.487	0.525 ± 0.487	0.525 ± 0.487
Model-Free	0.000	0.550 ± 0.415	0.350 ± 0.450	0.450 ± 0.472	0.225 ± 0.370	0.300 ± 0.400	0.350 ± 0.421	0.450 ± 0.415
Model-Free Tightened	0.000	0.600 ± 0.374	0.350 ± 0.450	0.500 ± 0.447	0.275 ± 0.402	0.425 ± 0.396	0.375 ± 0.415	0.425 ± 0.426
Post-Processing	0.550 ± 0.350	0.750 ± 0.335	0.550 ± 0.350	0.600 ± 0.300	0.400 ± 0.374	0.425 ± 0.396	0.550 ± 0.350	0.725 ± 0.295
Post-Processing Tightened	0.700 ± 0.332	0.700 ± 0.332	0.750 ± 0.403	0.750 ± 0.403	0.600 ± 0.300	0.600 ± 0.300	0.700 ± 0.292	0.700 ± 0.292

dynamic feasibility in diffusion-based trajectory planning. By guiding the reverse diffusion process with discrete-time CBF and CLF, our approach steers trajectory generation toward safe, goal-directed regions. This guidance provides high-quality warm starts for a subsequent MPC. Our framework’s effectiveness is demonstrated through an iterative-projection scheme where the MPC refines the trajectory at each denoising step, providing robust refinement and enforcing strict safety and dynamic constraints. Through extensive simulations in environments with both static and dynamic obstacles, alongside a sim-to-real experiment on a physical Franka manipulator, we have shown that D-SafeMPC significantly improves safety, task success rates, and planning efficiency compared to state-of-the-art baselines.

Despite these advancements, our framework has limitations. It may not scale well to high-dimensional state spaces (e.g., visual inputs), as MPC optimization becomes computationally prohibitive. Additionally, positioning inaccuracies in early time steps can propagate recursively, since the diffusion model conditions trajectory generation on the current state, potentially destabilizing subsequent predictions.

Future work will focus on extending this framework to systems with more complex, higher-dimensional state spaces while addressing scalability and error-propagation challenges.

REFERENCES

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 6840–6851.
- [2] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, “Planning with diffusion for flexible behavior synthesis,” in *ICML*, 2022.
- [3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, 2025.
- [4] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, “Motion planning diffusion: Learning and planning of robot motions with diffusion models,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.
- [5] J. Carvalho, A. T. Le, P. Kicki, D. Koert, and J. Peters, “Motion planning diffusion: Learning and adapting robot motion planning with diffusion models,” *IEEE Transactions on Robotics*, 2025.
- [6] W. Xiao, T.-H. Wang, C. Gan, R. Hasani, M. Lechner, and D. Rus, “Safediffuser: Safe planning with diffusion probabilistic models,” in *The Thirteenth International Conference on Learning Representations*, 2023.
- [7] V. Kurtz and J. W. Burdick, “Equality constrained diffusion for direct trajectory optimization,” in *American Control Conference (ACC)*. IEEE, 2025, pp. 535–540.
- [8] K. Mizuta and K. Leung, “CoBL-Diffusion: Diffusion-based conditional robot planning in dynamic environments using control barrier and Lyapunov functions,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 13 801–13 808.
- [9] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, “Control barrier function based quadratic programs for safety critical systems,” *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [10] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, “Control barrier functions: Theory and applications,” in *2019 18th European Control Conference (ECC)*, 2019.
- [11] U. Eren, A. Prach, B. B. Koçer, S. V. Raković, E. Kayacan, and B. Açikmeşe, “Model predictive control in aerospace systems: Current state and opportunities,” *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 7, pp. 1541–1566, 2017.
- [12] A. Amer, M. Mehndiratta, J. le Fevre Sejersens, H. X. Pham, and E. Kayacan, “Visual tracking nonlinear model predictive control method for autonomous wind turbine inspection,” in *21st International Conference on Advanced Robotics (ICAR)*, 2023, pp. 431–438.
- [13] A. Amer, M. Mehndiratta, Y. Brodskiy, and E. Kayacan, “Empowering autonomous underwater vehicles using learning-based model predictive control with dynamic forgetting Gaussian processes,” *IEEE Transactions on Control Systems Technology*, vol. 33, no. 5, pp. 1913–1920, 2025.
- [14] R. Römer, A. v. Rohr, and A. Schoellig, “Diffusion predictive control with constraints,” in *Proceedings of the 7th Annual Learning for Dynamics & Control Conference*, vol. 283. PMLR, 2025, pp. 791–803.
- [15] G. Zhou, S. Swaminathan, R. V. Raju, J. S. Guntupalli, W. Lehrach, J. Ortiz, A. Dedieu, M. Lázaro-Gredilla, and K. Murphy, “Diffusion model predictive control,” *arXiv preprint arXiv:2410.05364*, 2024.
- [16] G. Giannone, A. Srivastava, O. Winther, and F. Ahmed, “Aligning optimization trajectories with diffusion models for constrained design generation,” *Advances in neural information processing systems*, vol. 36, pp. 51 830–51 861, 2023.
- [17] R. Römer, L. Brunke, M. Schuck, and A. P. Schoellig, “Safe offline reinforcement learning using trajectory-level diffusion models,” in *ICRA 2024 Workshop—Back to the Future: Robot Learning Going Probabilistic*, 2024.
- [18] A. Agrawal and K. Sreenath, “Discrete control barrier functions for safety-critical control of discrete systems with application to bipedal robot navigation,” in *Robotics: Science and Systems*, vol. 13. Cambridge, MA, USA, 2017, pp. 1–10.
- [19] M. Ahmadi, A. Singletary, J. W. Burdick, and A. D. Ames, “Safe policy synthesis in multi-agent pomdps via discrete-time barrier functions,” in *2019 IEEE 58th Conference on Decision and Control (CDC)*. IEEE, 2019, pp. 4797–4803.
- [20] A. Q. Nichol and P. Dhariwal, “Improved denoising diffusion probabilistic models,” in *International conference on machine learning*. PMLR, 2021, pp. 8162–8171.
- [21] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, “Variational inference: A review for statisticians,” *Journal of the American Statistical Association*, vol. 112, no. 518, p. 859–877, Apr. 2017.
- [22] X. Jia, D. Blessing, X. Jiang, M. Reuss, A. Donat, R. Lioutikov, and G. Neumann, “Towards diverse behaviors: A benchmark for imitation learning with human demonstrations,” *arXiv preprint arXiv:2402.14606*, 2024.
- [23] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2012, pp. 5026–5033.
- [24] P. Dhariwal and A. Nichol, “Diffusion models beat GANs on image synthesis,” in *Advances in Neural Information Processing Systems*, vol. 34. Curran Associates, Inc., 2021, pp. 8780–8794.